

6. Abstract Factory Design Pattern in Java

6.1 Introduction

The **Abstract Factory Pattern** is a **creational design pattern** from the Gang of Four (GoF). It provides an interface for creating **families of related or dependent objects** without specifying their concrete classes.

6.2 Problem it Solves

When your code needs to work with various families of related products (UI elements, services, etc.), but shouldn't depend on their concrete classes, the Abstract Factory helps isolate that dependency.

6.3 What is Abstract Factory Pattern?

- It provides an interface to create families of related objects
- Concrete factories implement this interface
- Client code uses the abstract factory without knowing concrete types

6.4 Real-world Analogy

Imagine you're developing software for multiple operating systems. You want to create GUI elements (buttons, checkboxes, etc.) for each OS without hardcoding each version:

- **AbstractFactory:** UIFactory
- **ConcreteFactory:** WindowsUIFactory, MacUIFactory
- **Product Interfaces:** Button, Checkbox
- **Concrete Products:** WindowsButton, MacButton, etc.

6.5 Basic UML and Class Structure

AbstractFactory

- + createButton()
- + createCheckbox()

ConcreteFactoryA

- + createButton() -> ButtonA
- + createCheckbox() -> CheckboxA

ConcreteFactoryB

- + createButton() -> ButtonB
- + createCheckbox() -> CheckboxB

Client

+ uses AbstractFactory interface only

6.6 Step-by-Step Examples

Example1: Beginner: GUI Toolkit

```
// Abstract products
package com.mannemlokesk;

interface Button {
    void render();
}

interface Checkbox {
    void render();
}

// Concrete products for Windows
class WindowsButton implements Button {
    public void render() {
        System.out.println("Rendering Windows Button");
    }
}

class WindowsCheckbox implements Checkbox {
    public void render() {
        System.out.println("Rendering Windows Checkbox");
    }
}

// Concrete products for Mac
class MacButton implements Button {
    public void render() {
        System.out.println("Rendering Mac Button");
    }
}

class MacCheckbox implements Checkbox {
    public void render() {
        System.out.println("Rendering Mac Checkbox");
    }
}

// Abstract factory
interface UIFactory {
    Button createButton();

    Checkbox createCheckbox();
}

// Concrete factories
class WindowsUIFactory implements UIFactory {
    public Button createButton() {
        return new WindowsButton();
    }

    public Checkbox createCheckbox() {
```

```

        return new WindowsCheckbox();
    }
}

class MacUIFactory implements UIFactory {
    public Button createButton() {
        return new MacButton();
    }

    public Checkbox createCheckbox() {
        return new MacCheckbox();
    }
}

// Client
class Application {
    private Button button;
    private Checkbox checkbox;

    public Application(UIFactory factory) {
        this.button = factory.createButton();
        this.checkbox = factory.createCheckbox();
    }

    public void renderUI() {
        button.render();
        checkbox.render();
    }
}

public class Example1GUIMain {
    public static void main(String[] args) {
        UIFactory factory = new MacUIFactory();
        Application app = new Application(factory);
        app.renderUI();
    }
}

```

Output:

```

Rendering Mac Button
Rendering Mac Checkbox

```

Example3: Intermediate: Cloud Service Provider

```

package com.mannemlokes;

interface Storage {
    void upload(String file);
}

interface CDN {
    void distribute(String content);
}

class AWSStorage implements Storage {
    public void upload(String file) {
        System.out.println("Uploading to AWS S3: " + file);
    }
}

```

```
    }  
}  
  
class AzureStorage implements Storage {  
    public void upload(String file) {  
        System.out.println("Uploading to Azure Blob: " + file);  
    }  
}  
  
class AWSCdn implements CDN {  
    public void distribute(String content) {  
        System.out.println("Distributing via AWS CloudFront: " + content);  
    }  
}  
  
class AzureCdn implements CDN {  
    public void distribute(String content) {  
        System.out.println("Distributing via Azure CDN: " + content);  
    }  
}  
  
interface CloudFactory {  
    Storage createStorage();  
  
    CDN createCDN();  
}  
  
class AWSFactory implements CloudFactory {  
    public Storage createStorage() {  
        return new AWSStorage();  
    }  
  
    public CDN createCDN() {  
        return new AWSCdn();  
    }  
}  
  
class AzureFactory implements CloudFactory {  
    public Storage createStorage() {  
        return new AzureStorage();  
    }  
  
    public CDN createCDN() {  
        return new AzureCdn();  
    }  
}  
  
public class Example2CloudApp {  
    public static void main(String[] args) {  
        CloudFactory factory = new AWSFactory();  
        Storage storage = factory.createStorage();  
        CDN cdn = factory.createCDN();  
  
        storage.upload("project.zip");  
        cdn.distribute("video.mp4");  
    }  
}
```

Output:

```
Uploading to AWS S3: project.zip
Distributing via AWS CloudFront: video.mp4
```

6.7 Pros and Cons

 Pros:

- Groups related products together
- Ensures product consistency
- Makes swapping product families easy

 Cons:

- Increases number of classes/interfaces
- Complex when adding new product types

6.8 When to Use / Not to Use

Use When:

- You need to enforce consistency across families
- You expect multiple product variants (themes, OS, environments)

Avoid When:

- Product family is small or fixed
- You don't expect product combinations to change

6.9 Summary

Concept	Description
Factory Interface	Defines methods to create related objects
Concrete Factories	Implement family-specific creation
Abstract Products	Interface shared across families
Concrete Products	Implementations for each factory variant

The Abstract Factory Pattern allows encapsulation of object families and ensures that clients remain independent of specific product implementations.